

微机系统与接口——

第四章 8086汇编语言程序设计

东南大学信息科学与工程学院



王东明



1

汇编语言概述

2

汇编基本语法

3

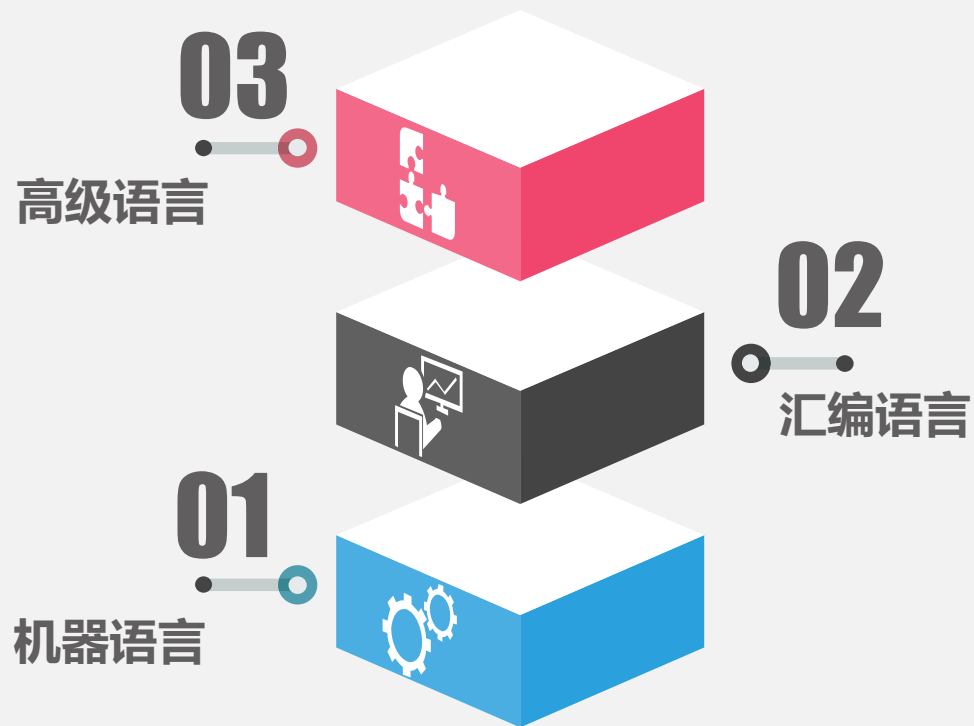
DOS中断调用

汇编语言概述

◆ 概述

◆ Hello World

程序设计语言



汇编语言

汇编语言

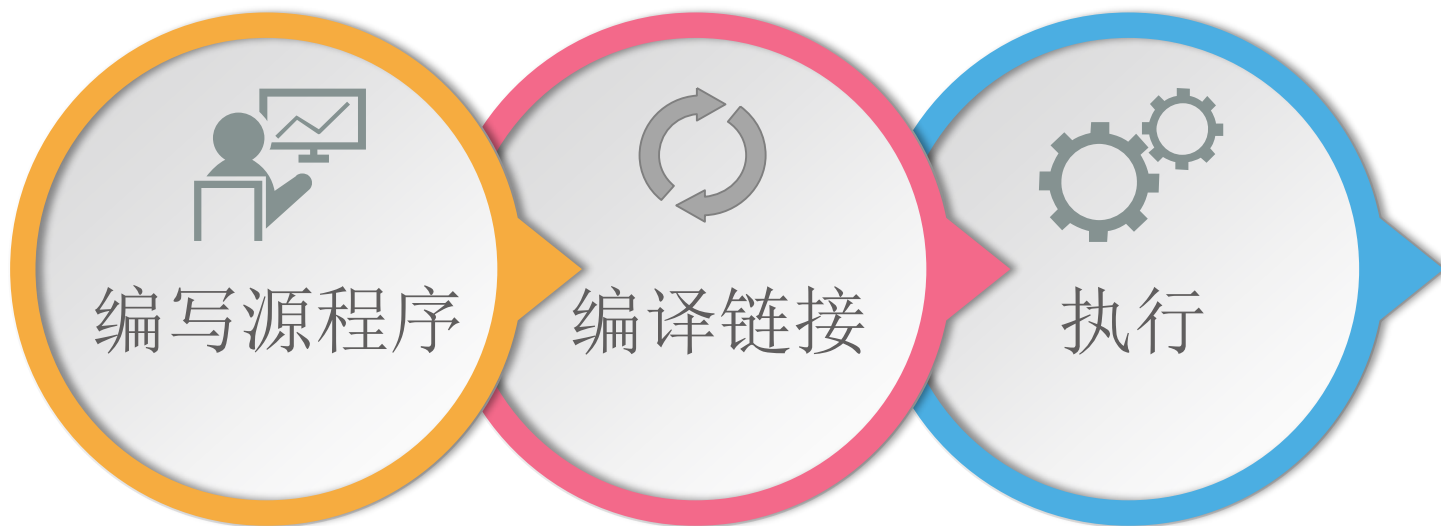
汇编语言是用指令的助记符、符号地址、标号等书写程序的语言,简称符号语言。

- 优点：易读、易写、易记。
- 缺点：不能像机器语言那样为计算机所直接识别，也不如高级语言那样具有很好的通用性和可移植性。

特点



从汇编语言源程序到执行的过程



汇编语言概述

◆ 概述

◆ Hello World

DATA SEGMENT

```
STR DB 'Hello world!', 0DH, 0AH, '$'  
DATA ENDS
```

定义数据段

```
SSEG SEGMENT PARA STACK 'STACK'  
DB 50 DUP(0)  
SSEG ENDS
```

定义栈段

CODE SEGMENT

```
ASSUME CS: CODE, DS: DATA, SS: SSEG  
START:
```

```
    MOV AX, DATA  
    MOV DS, AX  
    MOV AX, SSEG  
    MOV SS, AX  
    MOV DX, OFFSET STR  
    MOV AH, 9  
    INT 21H  
    MOV AH, 4CH  
    INT 21H
```

```
CODE ENDS
```

```
END START
```

定义代码段

定义代码结束



步骤1

编辑hello.asm

masm hello.asm

步骤2

步骤3

link hello

执行 hello.exe

步骤4



DATA SEGMENT

```
STR DB 'Hello world!', 0DH, 0AH, '$'  
DATA ENDS
```

数据段

汇编语言程序是按段来组织程序和数据。

```
SSEG SEGMENT PARA STACK 'STACK'  
DB 50 DUP(0)  
SSEG ENDS
```

栈段

程序中的段称为逻辑段，汇编连接后被映射到物理段中。

CODE SEGMENT

```
ASSUME CS: CODE, DS: DATA, SS: SSEG  
START:
```

每个段都有一个名字叫段名，以SEGMENT作为段的开始，以ENDS作为段的结束。这两者伪指令前面都要冠以相同的名字。

注意ENDS和END的作用区别。

START标号要有。

```
INT 21H
```

```
MOV AH, 4CH
```

```
INT 21H
```

```
CODE ENDS  
END START
```

整个源程序必须以END语句来结束，通知汇编程序停止汇编。END后面的标号表示该程序执行时起始地址。



伪指令语句

DATA **SEGMENT**

STR **DB** 'Hello world!', 0DH, 0AH, '\$'

DATA **ENDS**

SSEG **SEGMENT** **PARA** **STACK** 'STACK'

DB 50 **DUP**(0)

SSEG **ENDS**

CODE **SEGMENT**

ASSUME **CS:** **CODE**, **DS:** **DATA**, **SS:** **SSEG**

START:

指令语句

MOV **AX**, **DATA**

MOV **DS**, **AX**

MOV **AX**, **SSEG**

MOV **SS**, **AX**

MOV **DX**, **OFFSET** **STR**

MOV **AH**, 9

INT 21H

MOV **AH**, 4CH

INT 21H

伪指令语句

CODE **ENDS**

END **START**





1

汇编语言概述

2

汇编基本语法

3

DOS中断调用

汇编语句

指令语句

指令语句是一种执行性语句, 汇编程序将为之产生一一对应的机器目标代码。

伪指令语句

伪指令又称为伪操作, 是一种说明性语句, 它是在对源程序汇编期间由汇编程序处理的操作。

- 每一条指令都对应一种CPU操作。
- 伪指令可完成如处理器选择、定义程序模式、定义数据、分配存储区、指示程序结束等功能, 本身不生成目标代码。

注意



伪指令语句格式

名字

伪指令助记符

参数表

;注释

- 伪指令助记符，必须有，其它视情况可有可无。
- 名字：变量名、符号常量名字、段名、过程名等。
- 参数表：伪指令的参数。

注意



伪指令分类

数据定义

- DB DW
- DD DT
- DUP
- \$

符号定义

- EQU
- =
- LABEL

段定义

- SEGMENT
- ENDS
- ASSUME
- ORG AT
- PUBLIC等

过程定义

- PROC
- ENDP
- NEAR
- FAR
- END



伪指令语句

◆ 数据定义*****

◆ 符号定义

◆ 段定义

◆ 过程定义

数据定义伪指令

汇编时，数据定义伪指令完成数据类型定义及存储器分配等功能。

变量名

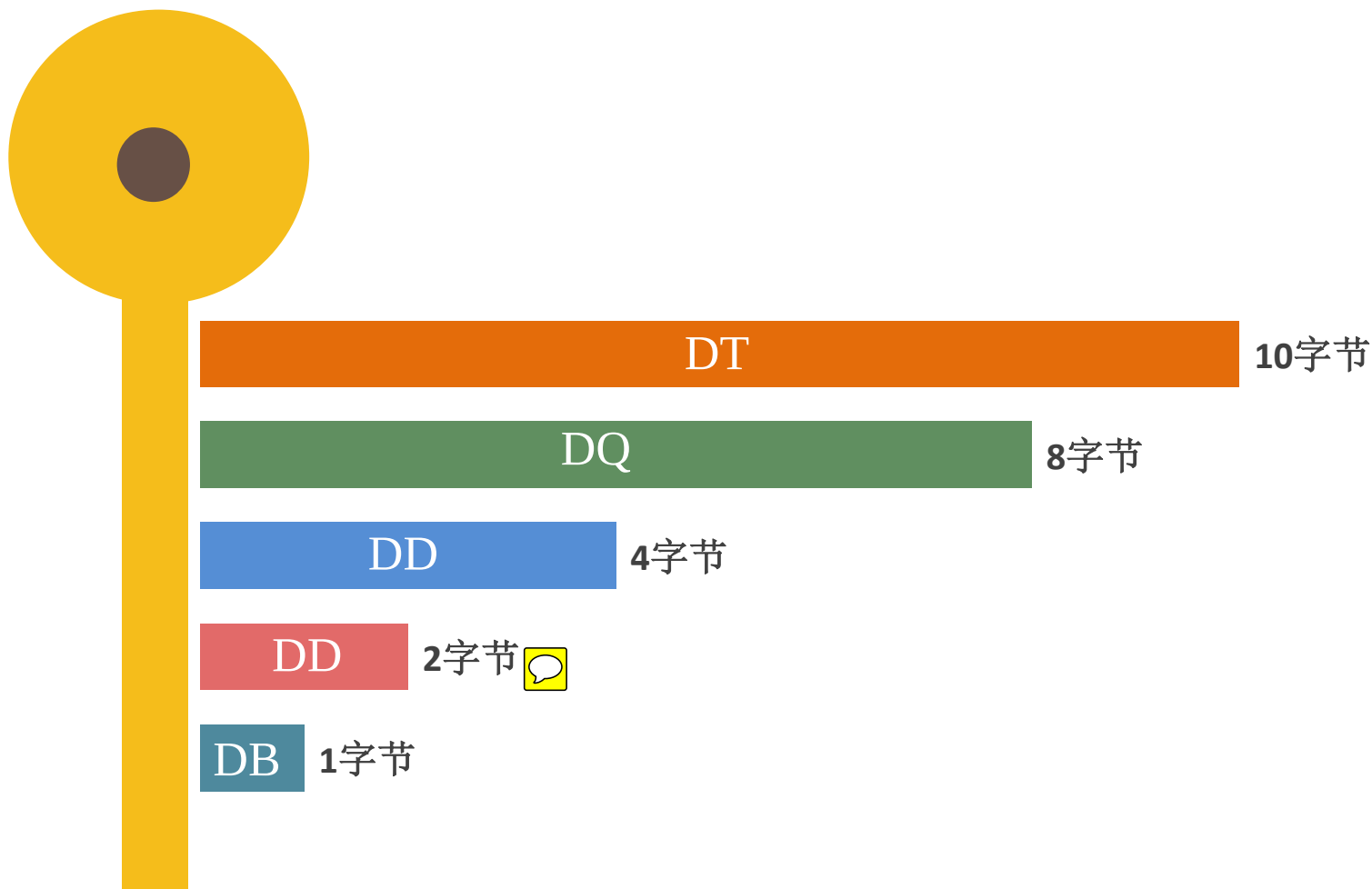
助记符

操作数, ..., 操作数,

- 伪指令：字节**DB**、字**DW**、双字**DD**、四字**DQ**、五字**DT**。
- **变量**是操作数地址的符号化表示。汇编程序将操作数第一个字节的偏移地址赋于该变量。后面操作数地址依次按变量类型得到。
- 操作数的形式可以是常量，数值表式，地址表达式，？，DUP。



数据定义伪指令



实验2：数据定义伪指令

```
DATA1 DW 12H, 2+3, ?
```

```
DATA2 DB 12H, -1, 0F5H, 12, ?
```

```
MOV AX, DATA1
```

```
MOV BX, DATA1+2
```

```
MOV CH, DATA1 ✗
```

类型不一致

```
MOV DL, DATA2+1
```

```
MOV CL, [DATA2+3]
```

```
MOV SI, DATA2 ✗
```

类型不一致

```
MOV CH, BYTE PTR DATA1
```

```
MOV SI, WORD PTR DATA2
```

? 表示此项数据无确定初值，数据类型仍由定义命令给出。



实验2：字符串定义

DB是唯一能定义长度大于2的字符串的伪指令，字符串用' '括起来，串中的每字符用相应ASCII码表示。

DATA3 DB 'SEU'

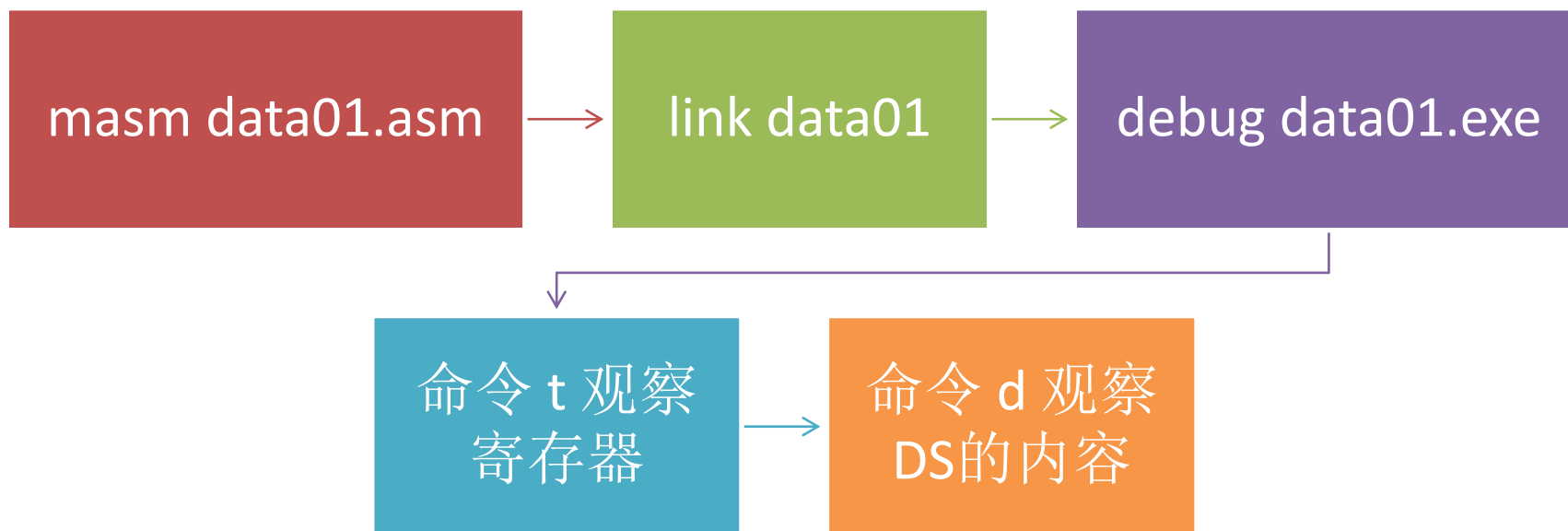
DATA4 DB 'AB'

DATA5 DW 'AB'

53H	低地址
45H	
55H	
41H	高地址
42H	
42H	
41H	



实验2的操作步骤



DUP伪指令

n

DUP

(操作数, ..., 操作数)

DW 100 DUP (?)

占用 200 字节

DD 5 DUP (?, 12, 3 DUP (12,5))

占用 160 字节

DB 3 DUP ('SEU', 'NJU', 3 DUP (?))

占用 27 字节



变量的属性

段属性

- 表示于该变量相对应的数据区所在段的段地址。

偏移值属性

- 表示该变量与段起始地址相距的字节数。

类型属性

- 表示地址所指数据区中数据项的存取单位是字节、字、双字、四字还是十字。



数据定义伪指令：地址表达式

- 数据定义时，出现在地址表达式中的变量表示该变量的偏移地址，用根据偏移地址计算出的地址表达式的值初始化相应的存储单元。
- 格式1： **DW 地址表达式**。利用该地址表达式的计算值初始化相应的存储字。
- 格式2： **DD 地址表达式**。地址表达式计算结果对应的段地址（高位字）和偏移地址（低位字）来初始化相应的两个存储字。



回顾C语言，这是不是定义了一个指针变量？



数据定义伪指令：地址表达式

FOO SEGMENT AT 55H

ZERO DB 2AH

ONE DW ONE

TWO DD TWO

THREE DD THREE+2

FOUR DW FOUR+2

FIVE DW FIVE-ONE

SIX DB \$-ZERO

FOO ENDS

2AH

55H : 0 ZERO

01H

55H : 1 ONE

00H

03H

55H : 3 TWO

00H

55H

00H

09H

55H : 7 THREE

00H

55H

00H

55H : 0AH



数据定义伪指令：地址表达式

FOO SEGMENT AT 55H

ZERO DB 0

ONE DW ONE

TWO DD TWO

THREE DD THREE+2

FOUR DW FOUR+2

FIVE DW FIVE-ONE

SIX DB \$-ZERO

FOO ENDS

0DH

55H : 0BH FOUR

00H

0CH

55H : 0DH FIVE

00H

0FH

55H : 0FH SIX

- 操作数为地址表达式时伪操作命令只能是DD和DW。
- 地址表达式中两个变量只能相减。
- \$是当前存储单元地址。



地址表达式：指令和伪指令中的区别

FOUR **DW** FOUR+2
FIVE **DW** FIVE-ONE

MOV AX, FOUR+2
MOV AX, FIVE-ONE

① 变量+常数

- 对数据定义伪指令，变量以地址的属性参与初始化。
- 在指令中表示存储器寻址，取其值，**取偏移地址用专门指令LEA或OFFSET伪指令。**

② 变量相减

- 指令和伪指令结果相同，结果没有地址属性，是一个数值，它表示两变量间相距的字节数。
- 它在汇编时计算。



伪指令语句

- ◆ 数据定义
- ◆ 符号定义
- ◆ 段定义
- ◆ 过程定义

符号定义伪指令

- 汇编中所有变量名、标号名、过程名、指令助记符，寄存器名统称为**符号**。
- 符号定义伪指令对符号重新命名或定义新的类型属性。

名字

EQU 或 =

表达式

变量名/
标号名

LABEL

类型

不分配存储空间，仅是给符号赋值或为变量/标号指定其类型。



EQU

为常量定义符号

ONE EQU 1

SUM EQU ONE+2

定义新的类型属性并重命名

FIRSTW EQU WORD PTR STR1

LABEL2 EQU FAR PTR LABEL1

MOV AL, SUM

MOV AX, FIRSTW

JMP LABEL2



MOV AL, 3

MOV AX, WORD PTR STR1

JMP FAR PTR LABEL1



EQU

为存储单元命名

XYZ EQU [BP+3]

A EQU ARR[BX][SI]

为符号定义新名字

COUNT EQU CX

LD EQU MOV

MOV XYZ, AL

MOV AL, A

LD COUNT, AX



MOV [BP+3], AL

MOV AL, ARR[BX][SI]

MOV CX, AX



= 和 LABEL伪指令

- = 与 EQU 作用基本相同，区别在于： ' = ' 伪操作允许重复定义。 ' = ' 一般习惯用于定义符号常量。
- LABEL可使指向同位置的不同变量或标号具有不同类型属性。

变量 ARRAY_BYTE 和 ARRAY_WORD、标号 SUBF和SUBN，具有相同的段值属性和偏移值属性，只是类型属性不同。

定义新的类型属性并重命名

ARRAY_BYTE LABEL BYTE

ARRAY_WORD DW 9

SUBF LABEL FAR

SUBN: SUB AX,[BX]



MOV AL, ARRAY_BYTE MOV AL, ARRAY_BYTE+1



伪指令语句

- ◆ 数据定义
- ◆ 符号定义
- ◆ 段定义
- ◆ 过程定义

段定义伪指令

段名

SEGMENT

定位类型

组合类型

'类别'

对数据段和堆栈段，段中语句一般是变量定义，对代码段是指令语句。

段名

ENDS



定位类型：确定逻辑段的边界

PARA

- 逻辑段从一个节(16个字节)的边界开始，即段起始地址应能被16整除。物理地址XXXX0H。默认。

BYTE

- 逻辑段从字节边界开始，即段可从任何地址开始。

WORD

- 逻辑段从字边界开始。即段起始地址必须是偶数。

PAGE

- 逻辑段从页边界开始。256字节称为一页，故段的起始物理地址XXX00H。



组合类型：不同模块中同名段的组合方式

PUBLIC

- 所有此类型的同名段组合成一个逻辑段，公用一个段地址，运行时装入同一个物理段中。

COMMON

- 所有此类型的同名段具有相同的起始地址(覆盖)，共享相同的存储区域。

AT

- AT 数值表达式：按绝对地址定位，段地址就是表达式的值。

STACK

- 专用于说明堆栈段，组合方式同PUBLIC。



举例

类别：类别名必须放入单引号内，连接程序只使同类别的段发生关联，所有同类别的段被安排在连续的存储区域中。

```
SSEG SEGMENT PARA STACK 'STACK'
```

```
DB 50 DUP(0)
```

```
SSEG ENDS
```

- **PARA**表示段的起始地址位于可用的第一节，每节16个字节。组合类型为**STACK**。
- 这里有'**STACK**'，链接程序会自动设置exe可执行文件里初始栈指针**SP**，程序里不用自己设置，否则汇编会提示警告。



ASSUME伪指令：建立段和段寄存器关系

ASSUME

段寄存器名: 段名, 段寄存器名: 段名, ...

- 由于ASSUME伪指令只是指定某个段分配给哪一个段寄存器，它并不能把段地址装入段寄存器中。所以在代码段中，还必须把段地址装入相应的段寄存器中。
- 在程序中不需要用指令装入代码段的段地址，因为在程序初始化时，装入程序已将代码段的段地址装入CS寄存器了。



ORG

- ORG规定了段内的指令或数据存放的开始地址(偏移地址的初值)。
- 表达式的值即为开始地址，从此地址起连续存放程序或数据。

ORG

表达式

```
ORG 0010H
```

```
NUM DB '2591'
```

NUM的偏移地址为：10H



伪指令语句

- ◆ 数据定义
- ◆ 符号定义
- ◆ 段定义
- ◆ 过程定义

段定义伪指令

过程名

PROC

类型

代码

RET或RETF

如果子程序中不能正确使用堆栈而造成执行RET前SP并未指向进入子程序时的返回地址，则必然会导致运行出错。

过程名

ENDP

- 类型默认是NEAR，表示段内调用，FAR表示段间调用。
- 过程名是子程序入口地址的符号化表示。它的属性有段地址、偏移地址、类型（NEAR，FAR）。



END

- END后跟的表达式通常就是**程序第一条指令的标号**，指示程序的启动地址(要执行的第一条指令的地址)。

END

标号



指令语句

◆ 标号

◆ 操作数的表达式

指令语句格式

标号:

前缀

指令助记符

操作数表

;注释

- 标号：表示冒号'：'后面的指令所在的逻辑地址。
- 操作数的表达式：操作数中涉及一些汇编伪指令。

注意



标号

命名

- A-Z(不分大小写), 0-9, ?, @, ., _, \$, 不能以数字开头, 句号(.)只能作为首字符。
- 长度小于31个字符, 不能与保留字(指令助记符、伪指令、预定义符号等)重名, 不能重复定义。

标号的属性

段属性

- 表示于该标号所在代码区的段地址。

偏移值属性

- 表示该标号指向的指令的偏移地址。

类型属性

- 远标号或是近标号。



标号

- 默认是NEAR类型。在循环或条件转移指令中，所用标号类型必须为NEAR，并且目标标号与本转移指令之间的距离不能超过+127和-128字节范围。
- 无条件转移和调用指令在段间转移时采用FAR类型，在段内可以采用FAR或者NEAR类型。过程名也是一个标号。

- **变量和标号**的共同点是，都是地址的符号化表示。
- 变量作为存储器操作数被引用，它指向DS或ES的数据项，标号指向的是代码段某条指令。
- 变量类型是存取单位大小，标号类型是NEAR或FAR。

区别



指令语句

◆ 标号

◆ 操作数的表达式

操作数的表达式：四种类型

 ①	常量
 ②	数值表达式
 ③	变量
 ④	地址表达式

常量

常量：在汇编过程中已确定数值的量，包括数值常量和符号常量。

数值常量

- 用各种进位制形式表示，如
0AH, 0CH。

符号常量

- 采用``='或``EQU"给常量定义一个名字，用名字在汇编语句中表示对应的常量。



数值表达式

数值表达式指在汇编时能被计算出并产生数值的表达式。该表达式由常量以算术、逻辑和关系运算符连接而成。

算术运算符

- +、-、*、/、MOD、SHR、SHL。

逻辑运算符

- AND、OR、XOR、NOT。

关系运算符

- EQ, NE, LT, GT, LE, GE。
- 关系成立时结果为FFFFH，否则为0。



数值表达式：举例

AND **AX**, ((NUMB **LT** 5) **AND** 30) **OR** ((NMB **GE** 5) **AND** 20)

- NUMB为一个符号常量
- 当NUMB < 5, 指令为AND AX,30
- 当NUMB >= 5, 指令为AND AX,20



变量：分析运算符

取地址运算符

- **SEG**: 取变量/标号的段地址
- **OFFSET**: 取变量/标号的偏移地址

```
VAR DW 1, 2, 3, 4, 5
```

```
MOV BX, OFFSET VAR  
MOV AX, SEG VAR
```

取值运算符

- **TYPE**: 取变量的类型 (1, 2, 4)
- **LENGTH**: 取所定义变量的长度 (即变量中元素的个数)
- **SIZE**: 取所定义存储区的字节数 ($= \text{TYPE} * \text{LENGTH}$)

```
TYPE VAR      值为2  
LENGTH VAR    值为5  
SIZE VAR      值为10
```



地址表达式

- 变量或者标号加上或者减去某个结果为整数的数值表达式，其**结果仍为变量或者标号**。
- 同一段内的变量或者标号可以相减，其结果不是地址，而是一个数值，该**数值表示两者间相距的字节数**。

```
DATA1 DW 1, 2, 3, 4, 5, 6
```

```
DATA2 DW 6
```

```
MOV BX, DATA1+5
```

```
MOV AX, DATA1[SI]
```

```
MOV AX, DATA1[5]
```

```
MOV AX, [DATA1+5]
```

```
MOV CX, DATA2-DATA1
```





1

汇编语言概述

2

汇编基本语法

3

DOS中断调用

问题

怎么用汇编完成用户
与PC硬件的交互？

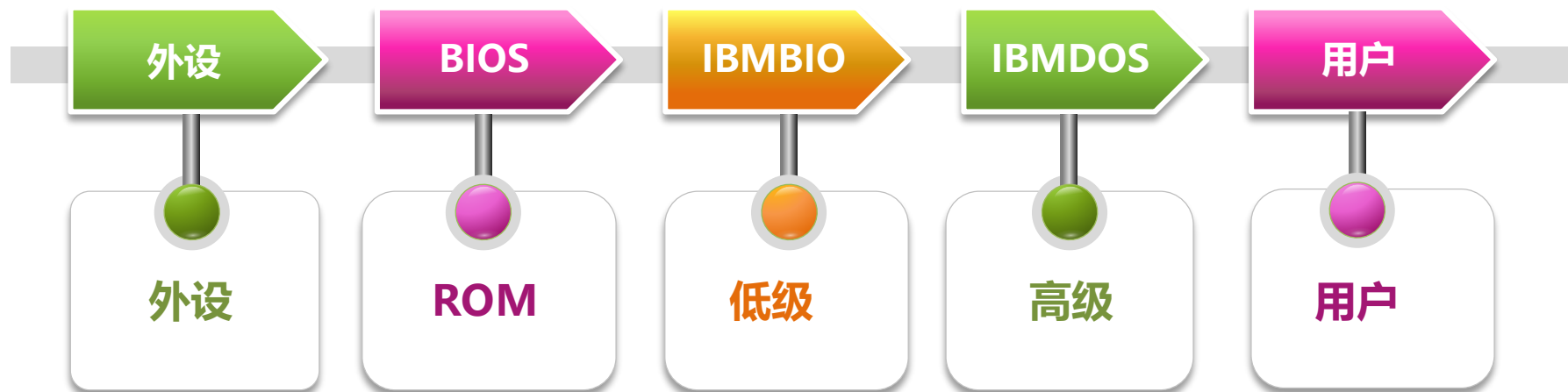


中断调用

- 为节省系统程序员编程量与优化程序结构，预先设计好了一系列的系统调用包括BIOS和DOS调用。
- 用户程序在调用这些系统服务程序时，不是用CALL命令，而是采用软中断指令INT n来实现。

BIOS和DOS调用

- BIOS是系统提供的基本输入输出例行程序，固化在ROM中，利用BIOS功能编写的程序简洁，可读性好，且易于移植。
- DOS是PC上重要的OS，它和BIOS一样包括有近百个设备管理、目录管理和文件管理程序，是一个功能齐全的中断程序集合。使用DOS操作比使用相应功能的BIOS更简易，且对硬件依赖性更少。



复习：INT n的功能

1

标志寄存器压栈

2

断点处的CS压栈、IP压栈

3

类型n对应的中断向量中4个字节传送给IP和CS

INT n

1

弹出IP、CS

2

弹出标志寄存器

IRET



DOS中断调用基本步骤

1 将调用参数装入指定的寄存器中

2 如需功能号，把它装入AH

3 按中断类型号调用中断

4 检查返回参数是否正确



常用DOS中断调用

键盘输入一字符并显示

类型号: 21H

功能号: 1

入口参数号: AL=输入字符的ASCII

显示一字符

类型号: 21H

功能号: 2

入口参数号: DL=待显示字符的ASCII码



常用DOS中断调用

字符串输入（回车做为输入结束符）

类型号：21H

功能号：0AH

入口参数号：DS：DX指向输入缓冲区



N1：程序员设定的最大键入的字符串长度，包括回车

N2：实际键入的字符数，不包括回车，系统自动写入

缓冲区格式要求



常用DOS中断调用

显示以 “\$” 结尾的字符串

类型号: 21H

功能号: 9

入口参数号: DS: DX指向字符串的首地址

返回DOS

类型号: 21H

功能号: 4CH



谢谢听讲

授课
教师

王东明

移动通信国家重点实验室



Tel: 13915982595



wangdm@seu.edu.cn



无线谷1-205

